# Protocolos de comunicaciones de alta velocidad

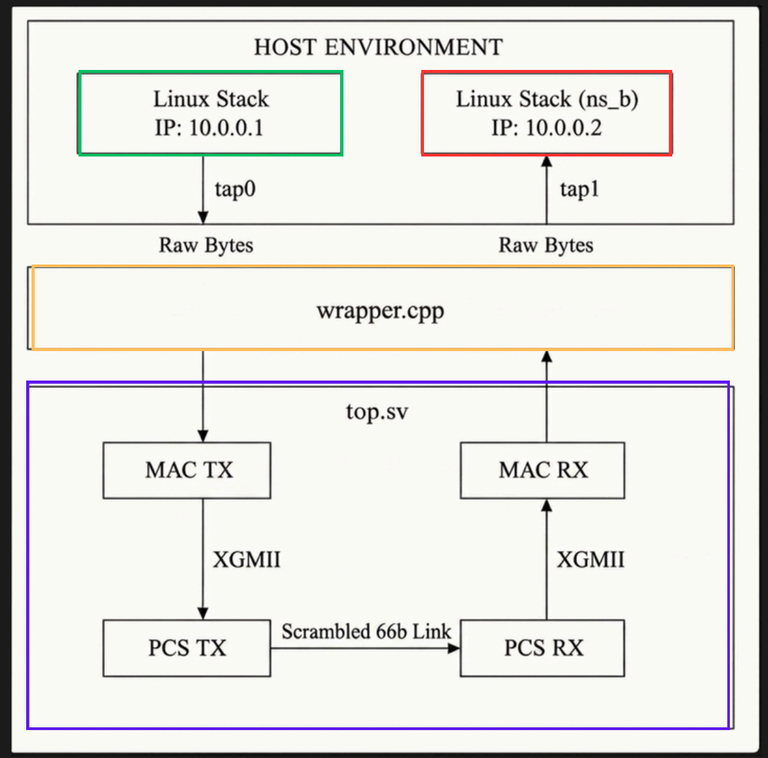
# Ethernet-phy

## Lab3 PCS 64b/66b Encoding & Scrambling

Fundación Fulgor

Facundo Guerrero

# Architecture Setup




- Consists of two virtual interfaces, Tap0 and Tap1, which can communicate with each other (for example, using the *ping* command).
- In between them, there is a *wrapper* that acts as a bridge. It reads the raw Ethernet frames from the TAP interfaces and injects them directly into the input pins of the *top.sv* circuit. At the same time, it records a .fst file for waveform visualization.
- The digital circuit *top.sv* implements the hardware PHY layer. It instantiates the MAC and connects it via the XGMII bus (64-bit data + 8-bit control) to the PCS 64b/66b Encoder/Decoder and the 58-bit LFSR Scrambler.
- In this case, Lab 3, the data inside the hardware is processed by a MAC layer, then XGMII, and encoded into 66-bit blocks by a PCS 64b/66b module, and converted into pseudo-random noise by a 58-bit LFSR Scrambler before being decoded at the receiver.
- Finally, the packets are analyzed using Wireshark and GTKWave.

# Architecture Setup

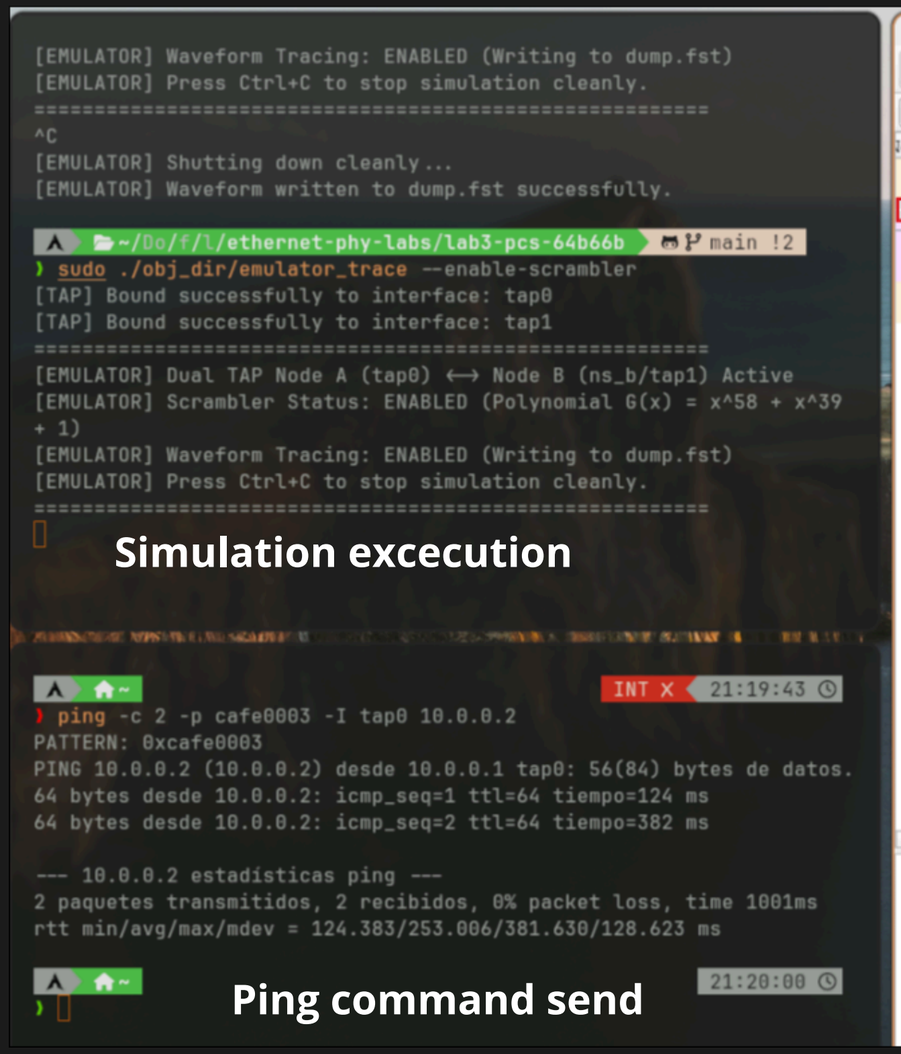
```
~/Documentos/fulgor/labs/ethernet-phy-labs/scripts  main
❯ sudo ./setup_netns.sh
[sudo] contraseña para facusu:
[NETNS] Cleaning up previous setup...
[NETNS] Creating namespace ns_b...
[NETNS] Creating TAP interfaces tap0 and tap1...
[NETNS] Moving tap1 into namespace ns_b...
[NETNS] Disabling IPv6 to suppress ICMPv6 multicast packets...
[NETNS] Assigning IP addresses...
[NETNS] Bringing interfaces UP...
------------------------------------------------
[NETNS] Setup Complete:
  * Host (Host NetNS): tap0 → 10.0.0.1/24 (IPv6 OFF)
  * Node B (ns_b NetNS): tap1 → 10.0.0.2/24 (IPv6 OFF)
------------------------------------------------
```

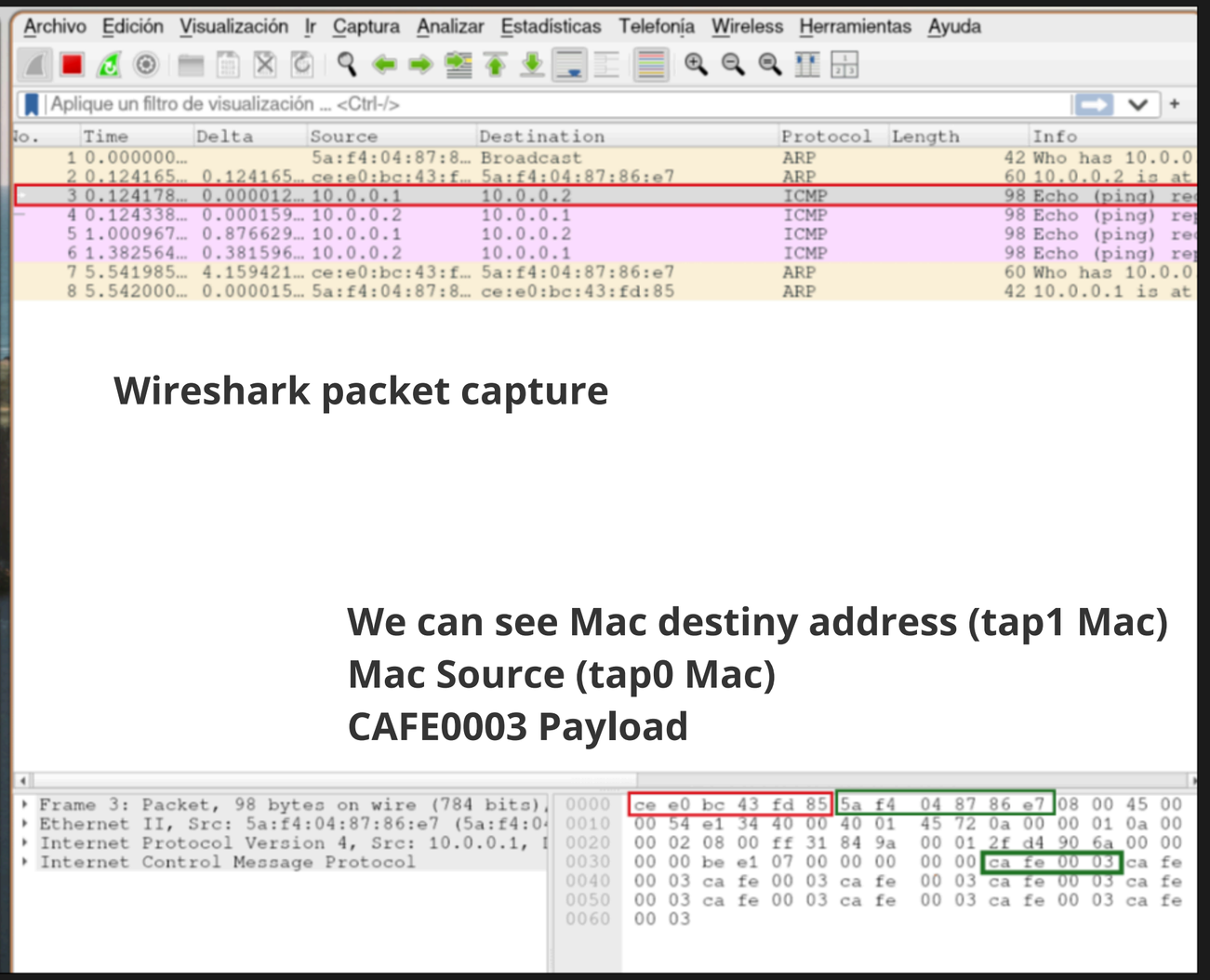
1. We run *setup_netns.sh* to create the TAPS

```
~/Documentos/fulgor/labs/ethernet-phy-labs/lab3-pcs-64b66b  main !1                                                
```

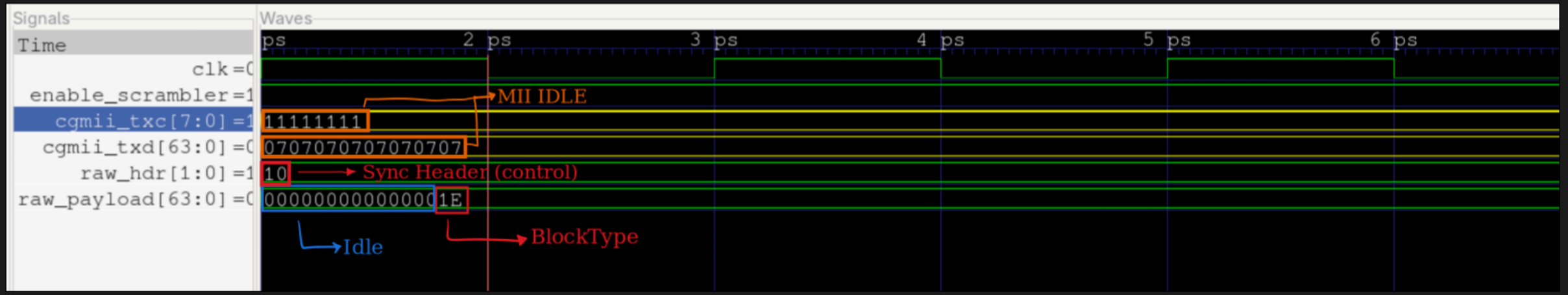
# Execution

It is executed simultaneously in three terminals.



# Analysis

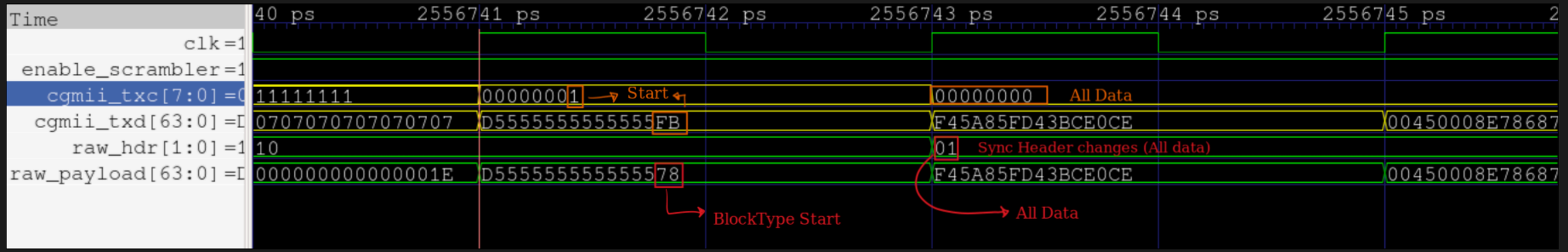



**XGMII fields indicate an Idle state**

*Sync Header = 10* **indicates the presence of at least one control character**

**We can see** *BlockType = 1E* **which indicates all characters in this payload are control characters. Specifically, they are Idles (represented with 00 becasue PCS 7-bit mapping).**

# Analysis

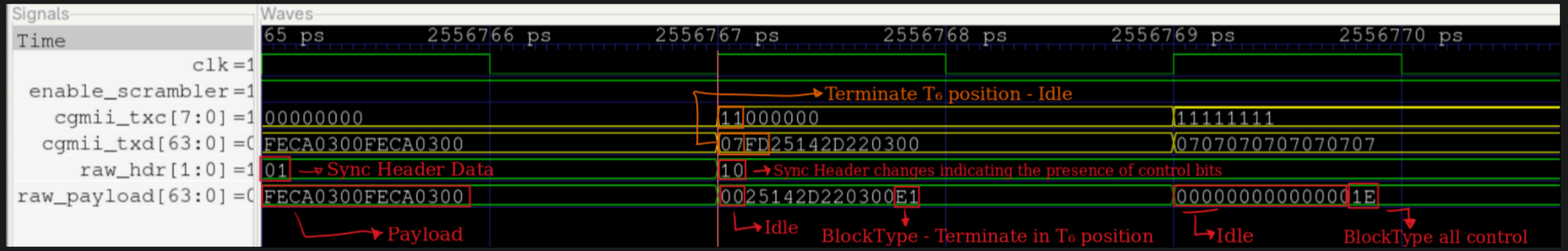



**XGMII fields indicate an Start state**

**We can see** *BlockType = 78* **which indicates the Start of Frame transmission.**
**The 8-bit Start control character** *(/S/ = 0xFB)* **is replaced by the** 78 Block Type**, followed by the 7 bytes of the Ethernet preamble (***D5555555555555***).**

**After this XGMII fields indicate all data frame, at this moment we can notice how** *SyncHeader* **turn to** *01* **indicates all characters in the block are data.**

# Analysis



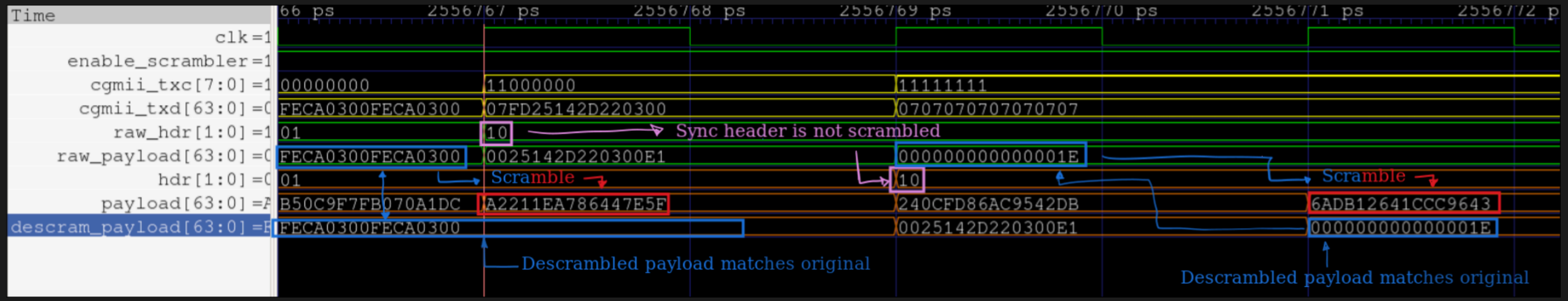

**We can see CAFE0003 payload**

**Then the XGMII indicates a block terminate control character at the $T_6$ position followed by an idle character.**

- *SyncHeader* **turns to** *10* **due to the presence of a control characters.**
- *BlockType = E1* **because the Terminate character is at the $T_6$ position. The** *0xFD* **character is not explicitly encoded on PCS payload because** *E1 BlockType* **provides all the necessary information.**

**Finally** *BlockType = 1E* **indicates that all characters in the block are control characters, all mapped to 00 (Idles).**

# Analysis




**Finally in this image we can observe:**

- The payload is changed on a scramble signal adding an pseaudoRandom noise, but is correctly reconstructed on the receiver side (descram_payload).

- Sync Header is never scrambled (it bypasses the scrambler).